

EPIC Scan Agent — Instance Profile

EPIC Scan Agent — Instance Profile / Spec Sheet

A point-in-time specification of the EC2 instance hosting the **EPIC - Self-hosted** Azure DevOps agent. For the step-by-step build procedure see [scan-agent-setup.md](#); for operational fixes see its Troubleshooting table.

- **Provisioned:** 2026-06-12
- **Provisioning model:** Terraform provisions the host; scan toolchain installed by hand over SSM (hand-built box)
- **Terraform stack:** EPIC AWS Resources/Scan Agent/ (local state, same pattern as Deploy Role)
- **Purpose:** runs SonarQube + Wiz scans, and .NET+SonarQube builds, for the EPIC pipeline

1. AWS — Account & Region

Item	Value
AWS account ID	514712703977
Region	us-west-2
Environment	dev

2. AWS — Compute (EC2)

Item	Value
Instance role	ADO self-hosted scan agent
Name tag	pge-epic-scan-agent-dev
Instance type	m5.large (2 vCPU, 8 GB RAM — OS reported ~7735 MB)
AMI	Latest Amazon Linux 2023 x86_64 (resolved dynamically via <code>data.aws_ami, al2023-ami-2023.*-x86_64</code>)
Architecture	x86_64 (amd64)
Kernel (observed)	6.18.33-63.124.amzn2023.x86_64
Private hostname (observed)	ip-10-90-0-28.us-west-2.compute.internal

Item	Value
Monitoring	enabled (detailed)
EBS optimized	true
IMDS	IMDSv2 required (http_tokens = required), hop limit 2, instance tags exposed
Termination protection	disabled

3. AWS — Storage

Item	Value
Root volume size	60 GB
Root volume type	gp3
Encryption	enabled (EBS encrypted at rest)

Toolchain, agent files, and the adoagent home all live on this root volume and **persist across reboot / stop-start**. Only instance **termination** loses them (rebuild from the runbook).

4. AWS — Networking

Item	Value
VPC	vpc-8c57a5f4 (shared EPIC nonprod VPC — same as epic-api)
Subnet	subnet-f9206980 (subnet epic-api's app server runs in; proven outbound egress)
Public IP	none
Security group	pge-epic-scan-agent-dev (created by the stack; not reused from epic-api)
Inbound rules	none (access is via SSM Session Manager, outbound-initiated)
Outbound rules	HTTPS 443 → 0.0.0.0/0 (ADO, SonarQube, Wiz, Secrets Manager, package repos)

Confirmed reachable egress targets:

- Azure DevOps — `dev.azure.com`, `*.vssps.visualstudio.com` (agent polls outbound)
- Internal SonarQube — `https://sonarqube.nonprod.pge.com` (server v2026.2.1.121354)
- NuGet — `api.nuget.org`
- Wiz — `wizcli.app.wiz.io`

5. AWS — IAM

Item	Value
Instance profile	pge-epic-scan-agent-dev-instance-profile (created by module)
IAM role	pge-epic-scan-agent-dev-ec2-role
Attached managed policy	AmazonSSMManagedInstanceCore (Session Manager access)
Custom policy	pge-epic-scan-agent-dev-secrets-read — only if scan_secret_arns is set; currently empty [], so not attached

Scan credentials currently come from ADO (SonarQube service connection + GV-account-access variable group for Wiz), **not** AWS Secrets Manager — so the agent needs no AWS secret access today.

6. AWS — Tagging (epic-pipeline-module-aws-tags)

Tag input	Value
appid	2102
owner	rhmg, def2, ghi3
notify	rhmg@pge.com, def2@pge.com, ghi3@pge.com
order	70056008
dataclassification	Internal
compliance	None
cris	Low
Extra tags	Role=ado-scan-agent, Pool=EPIC - Self-hosted

7. On-Instance — OS & Access

Item	Value
OS	Amazon Linux 2023
Package manager	dnf
Access method	AWS SSM Session Manager (no SSH, no key pair, no bastion)
SSM login user	ssm-user (unprivileged; sudo su - for root)

8. On-Instance — Installed Toolchain

All installed by hand per the runbook. Versions are as observed on 2026-06-12.

Tool	Version (observed)	Source	Purpose
git	(dnf latest)	dnf	source checkout — required ; agent detects at startup
libicu	(dnf latest)	dnf	.NET globalization dependency — required or dotnet aborts
Java (JRE)	Corretto 17.0.19	dnf java-17-amazon-corretto-headless	SonarScanner runs on the JVM
Node.js / npm	v18.20.8	dnf nodejs npm	jest/vitest coverage paths SonarQube reads
.NET SDK	10.0.301	dotnet-install.sh → /usr/share/dotnet (symlink /usr/local/bin/dotnet)	.NET builds + SonarQube dotnet mode
dotnet-sonarscanner	(global tool, optional)	dotnet tool install --global (as adoagent)	SonarQube dotnet mode (optional — SQ task downloads its own)
Wiz CLI	v0.109.15	wizcli.app.wiz.io → /usr/local/bin/wizcli	Wiz IaC/secrets/vuln scans
Base utils	—	dnf	curl, tar, gzip, unzip, rsync, which, jq

Note — SonarQube uses its own embedded Node 20 for JS/TS analysis (seen in scan logs: `/home/adoagent/.sonar/js/node-runtime/node`), independent of the dnf-installed Node 18. **Note — Wiz CLI v0.X is past End-of-Support (Apr 15 2026)**; migrating to v1.X is a separate pipeline-level change.

9. On-Instance — Azure DevOps Agent

Item	Value
Service user	adoagent (uid/gid 1002, non-root)
Agent install dir	/home/adoagent/myagent
Agent version	4.274.1 (Linux x64)
Agent name	scan-agent-01
ADO org	https://dev.azure.com/pgetech
Pool	EPIC - Self-hosted
Run mode	systemd service (<code>./svc.sh install adoagent</code>), enabled
systemd unit	<code>vsts.agent.pgetech.EPIC\x20\x2d\x20Self\x2dhosted.scan\x2dagent\x2d01.service</code> (escaped name — manage via <code>./svc.sh</code> , not raw <code>systemctl</code>)
Auto-recovery	Yes — restarts on reboot / stop-start automatically
Concurren cy	1 job at a time (single agent — see scaling note)
Status	Online / Idle (validated 2026-06-12)

10. PATH note for the agent user

`/home/adoagent/.bashrc` adds: `$PATH:/home/adoagent/.dotnet/tools:/usr/local/bin` (so dotnet global tools, the dotnet symlink, and wizcli resolve for agent jobs).

11. ADO-side dependencies (not on the instance)

Item	Value	Notes
Agent pool	EPIC - Self-hosted	exact name must match pipeline YAML
SonarQube service connection	named SonarQube	points at <code>https://sonarqube.nonprod.pge.com</code> ; uses a Global Analysis token
Wiz credentials	WIZ_CLIENT_ID / WIZ_CLIENT_SECRET	from GV-account-access variable group

12. Security posture

The safety built into this instance, by layer. Most of this is enforced in the Terraform stack (EPIC AWS Resources/Scan Agent/), so it is reproducible on rebuild — not manual hardening.

Network isolation

- **No inbound exposure.** The security group has **zero ingress rules**. Nothing on the internet or in the VPC can initiate a connection to this host.
- **No public IP, no SSH.** The instance has no public address and no key pair. There is no SSH daemon path in from outside.
- **Egress scoped to HTTPS.** Outbound is limited to TCP 443 — enough to reach ADO, SonarQube, Wiz, and package repos, but not arbitrary ports/protocols.
- **Private subnet.** Lives in the shared EPIC nonprod VPC on a private subnet; reaches the internet only via the subnet's managed egress path.
- **Outbound-only agent model.** The ADO agent *polls* Azure DevOps for work — ADO never connects in. This is why no inbound rule is needed and is the core reason the box can stay fully closed.

Access control

- **SSM Session Manager only.** All human/operator access is via AWS Systems Manager Session Manager — brokered through the AWS API, authenticated by IAM, and fully audit-logged. No long-lived SSH keys to manage, leak, or rotate.
- **IAM-gated shell.** Reaching the box requires AWS credentials with SSM permissions; access is governed by IAM policy, not by network position or a shared secret.

Identity & least privilege (IAM)

- **Scoped instance role.** The instance profile carries only `AmazonSSMManagedInstanceCore` (Session Manager) by default — no broad AWS permissions.

- **No standing secret access.** The optional Secrets Manager read policy is **not attached** (`scan_secret_arns` is empty), and even when used it is **scoped to specific secret ARNs**, not `secretsmanager:*`. The agent holds no AWS data-plane privileges today.
- **No deploy rights.** This agent only scans/builds; it never assumes `pge-epic-deployment-role` and cannot provision or modify infrastructure.

Instance hardening

- **IMDSv2 enforced.** `http_tokens = required` blocks the SSRF-style credential-theft vector that IMDSv1 is prone to; metadata hop limit is capped at 2.
- **Encryption at rest.** The root EBS volume is encrypted.
- **Detailed monitoring on,** EBS-optimized.

Credential handling

- **Scan secrets live in ADO, not on the box.** The SonarQube token (service connection) and Wiz credentials (GV-account-access variable group) are injected into pipeline jobs at run time by Azure DevOps and are not stored on the instance.
- **Registration PAT is single-use.** The PAT used to register the agent is needed only once and is revoked after registration — it does not persist on the host.
- **Non-root agent.** The ADO agent runs as the unprivileged `adoagent` user (uid 1002), never as root, limiting the blast radius of a compromised build/scan job.

Residual risks / things to watch

- **Egress is 0.0.0.0/0 on 443.** Open by destination (any HTTPS host), not locked to specific ADO/SonarQube/Wiz CIDRs. Tightening to an allow-list of endpoints would reduce exfil surface if ever required.
- **Build/scan jobs run arbitrary repo code.** A self-hosted agent executes whatever the pipeline tells it to (npm installs, dotnet builds, etc.). The non-root user + closed inbound + scoped IAM contain this, but it is the inherent trust model of any self-hosted CI agent — keep the toolchain patched and treat the box as semi-trusted.
- **Hand-built drift.** Manual toolchain installs aren't captured in an AMI; the runbook is the control. A baked/golden AMI would make the security baseline immutable and reproducible.

13. Known limitations / planned changes

- **Single agent** — concurrent scan-bearing pipelines queue. Planned: register multiple agents on this same box (`scan-agent-02`, etc.) for concurrency, bounded by the 8 GB RAM (SonarQube is memory-heavy; may require sizing up). See the multi-agent TODO.
- **SonarQube memory** — if large .NET solutions OOM on `m5.large`, bump `instance_type` to `m5.xlarge` (16 GB) in `tfvars` and `terraform apply`.
- **Wiz CLI v0.X** past End-of-Support — migrate to v1.X (separate pipeline change).
- **Hand-built** — this profile + the setup runbook are the only rebuild record; keep both current when patching or adding tooling.